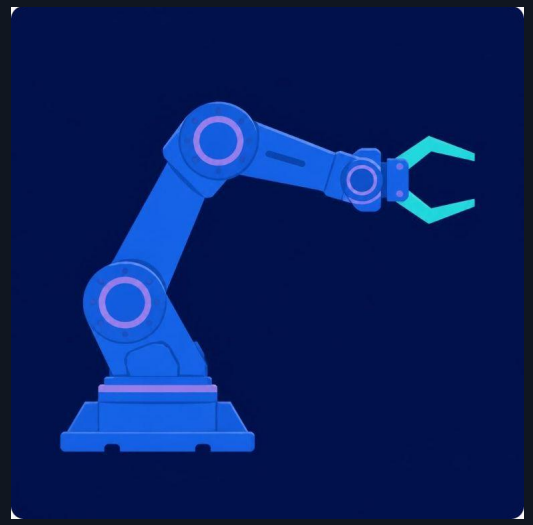
# ROS 2 com Python e Jupyter: Do Ambiente à Prática

ESP-004 | Tópicos Especiais em Engenharia

IFBA — PPG-ESP | 2025

*Robótica · Programação · Inteligência Artificial*





# Por que estudar ROS 2?

**O padrão da indústria e da pesquisa em robótica**

- 🤖 ROS 2 é usado em drones, carros autônomos, braços robóticos e robôs móveis
- 🌍 Mais de 3.000 pacotes disponíveis na comunidade
- 🔬 Base de projetos NASA, BMW, Amazon Robotics
- 📦 Esta disciplina usa **ROS 2 Jazzy** (Ubuntu 24.04 — LTS até 2029)

> *"Se você aprende ROS 2, você fala a língua da robótica moderna."*

# Visão Geral da Disciplina

**4 Notebooks + Apostila + Atividade Avaliativa**

Tabela: Notebook | Tema | O que você vai fazer

| Notebook | Tema | O que você vai fazer |
|---|---|---|
| 📓 NB-01 | Verificação do Ambiente | Testar Ubuntu, ROS 2 e Python |
| 📓 NB-02 | Fundamentos ROS 2 | Criar nós, publicar e receber mensagens |
| 📓 NB-03 | Jupyter Interativo | Controlar robô com sliders, treinar IA |
| 📓 NB-04 | Atividade Avaliativa | Implementar PatrolNode autônomo |

🔗 REPOSITÓRIO: GITHUB.COM/ESP004-PPGESP/NOTEBOOKS

# Onde está o material?

**Links para acessar notebooks e apostila**

📓 **NB-01** → github.com/esp004-ppgesp/notebooks/blob/main/nb01_verificacao_ambiente.ipynb

📓 **NB-02** → github.com/esp004-ppgesp/notebooks/blob/main/nb02_ros2_fundamentos.ipynb

📓 **NB-03** → github.com/esp004-ppgesp/notebooks/blob/main/nb03_jupyter_interativo.ipynb

📓 **NB-04** → github.com/esp004-ppgesp/notebooks/blob/main/nb04_atividade_avaliativa.ipynb

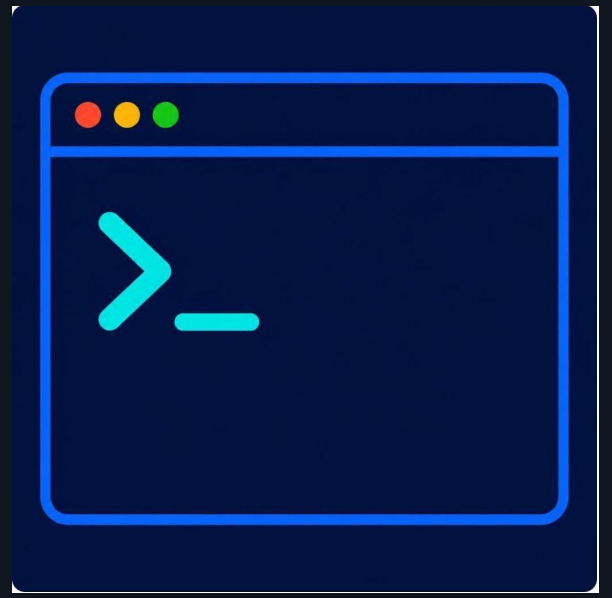
📖 **Apostila completa** → disponível no repositório como apostila_esp004.pdf

Os links dos notebooks aparecem em cada slide prático — você pode acessar diretamente pelo celular ou computador.

# Módulo 1: Ambiente de Trabalho

**O que você precisa instalar?**

- ✅ **Ubuntu 24.04 LTS** em dual-boot com Windows 11
- ✅ **ROS 2 Jazzy Jalisco** — versão LTS suportada até 2029
- ✅ **Python** com rclpy, numpy, matplotlib, pandas, torch, ipywidgets
- ✅ **JupyterLab** com kernel especial que carrega o ROS 2 automaticamente



📓 NB-01 — Verificação do Ambiente

🔗 github.com/esp004-ppgesp/notebooks/blob/main/nb01_verificacao_ambiente.ipynb

# Dual-boot: Ubuntu + Windows

**Como instalar Ubuntu sem apagar o Windows**

1. Baixar Ubuntu 24.04 LTS de ubuntu.com/download
2. Criar pendrive bootável com Rufus (formato ISO)
3. Desativar Fast Startup no Windows (Painel de Controle → Opções de Energia)
4. Desativar Secure Boot na BIOS/UEFI
5. Reduzir partição Windows (mínimo 50 GB para Ubuntu)
6. Instalar Ubuntu na partição livre — GRUB gerencia o boot duplo



⚠️ Antes de começar: faça backup dos seus dados!

# Instalando ROS 2 Jazzy

**3 comandos essenciais no terminal Ubuntu**

Adicionar repositório oficial ROS 2:

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
```

Instalar ROS 2 Jazzy Desktop completo:

```
sudo apt install ros-jazzy-desktop
```

Adicionar ao perfil (executa automaticamente):

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
```

✅ Após instalar: ros2 --version deve mostrar "jazzy"

# O Kernel Especial do Jupyter

**Por que não funciona com o kernel padrão?**

O ROS 2 precisa de variáveis de ambiente carregadas ANTES de qualquer código Python. O kernel padrão do Jupyter NÃO faz isso automaticamente.



Solução: criar um kernel que executa "source /opt/ros/jazzy/setup.bash" automaticamente.

O arquivo kernel.json fica em: ~/.local/share/jupyter/kernels/python3-ros2/kernel.json

Com o conteúdo:

```
argv: ["bash", "-c", "source /opt/ros/jazzy/setup.bash && python3 -m ipykernel_launcher -f
{connection_file}"]
display_name: "Python 3 (ROS2 Jazzy)"
language: "python"
```

📓 INSTRUÇÕES DETALHADAS: NB-01, SEÇÃO 1.3

# Módulo 2: Fundamentos de ROS 2

**Os 4 conceitos que você precisa dominar**

## Nó (Node)

Processo Python independente que faz uma tarefa específica

## Tópico (Topic)

Canal de comunicação — nós publicam e recebem mensagens

## Serviço (Service)

Comunicação requisição/resposta (chamada + retorno)

## Parâmetro (Parameter)

Configurações do nó que podem ser alteradas em tempo real

📓 NB-02 — Fundamentos ROS 2

🔗 github.com/esp004-ppgesp/notebooks/blob/main/nb02_ros2_fundamentos.ipynb

# Criando seu Primeiro Nó

**HelloNode — publicador com timer (NB-02, Exemplo 2)**

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
class HelloNode(Node):
    def __init__(self):
        super().__init__('hello_node')
        self.pub = self.create_publisher(String, '/esp004/hello', 10)
        self.timer = self.create_timer(1.0, self.callback)
```

```
def callback(self):
    msg = String()
    msg.data = "Olá do ROS 2!"
    self.pub.publish(msg)
```

A cada 1 segundo, o nó publica uma mensagem no tópico /esp004/hello

📓 Execute no: NB-02, Exemplo 2

# Publicador e Subscriber

**Como dois nós se comunicam via tópico (NB-02, Exemplo 3)**

Fluxo de comunicação: [HelloNode] → publica String → /esp004/hello → assina → [OuvidorNode]

```
class OuvidorNode(Node):
    def __init__(self):
        super().__init__('ouvidor')
        self.sub = self.create_subscription(String, '/esp004/hello', self.ouvir, 10)
    def ouvir(self, msg):
        self.get_logger().info(f"Recebi: {msg.data}")
```

📓 EXECUTE NO: NB-02, EXEMPLO 3

# Controlando Velocidade com Twist

**O tipo de mensagem mais importante em robótica móvel**

```
from geometry_msgs.msg import Twist
cmd = Twist()
cmd.linear.x  = 0.5   — m/s para frente (positivo) / ré (negativo)
cmd.angular.z = 0.3   — rad/s girar à esquerda (+) / direita (-)
publisher.publish(cmd)
```



Tabela: Campo | Significado | Unidade

| Campo | Significado | Unidade |
|---|---|---|
| linear.x | Velocidade linear (frente/ré) | m/s |
| angular.z | Velocidade angular (rotação) | rad/s |

📓 Veja a cinemática diferencial no: NB-02, Exemplo 5

# Módulo 3: Jupyter Interativo

**Controle em tempo real com sliders e botões**

O que você vai construir no NB-03:

- Painel de controle com slider de velocidade linear e angular
- Botão PARAR que zera o movimento instantaneamente
- Dashboard ao vivo com gráfico de trajetória em tempo real
- ObstacleNet — rede neural treinada para detectar obstáculos

> "Você controla o robô sem sair do Jupyter!"

📓 NB-03 — Jupyter Interativo

🔗 github.com/esp004-ppgesp/notebooks/blob/main/nb03_jupyter_interativo.ipynb

# Painel de Controle Interativo

**ipywidgets — NB-03, Seção 3.1**

```
import ipywidgets as widgets
sl_vlin = widgets.FloatSlider(min=-1.0, max=1.0, step=0.05, description='Linear (m/s):')
sl_vang = widgets.FloatSlider(min=-2.0, max=2.0, step=0.1, description='Angular (rad/s):')
btn_stop = widgets.Button(description='PARAR', button_style='danger')
def on_stop(_):
    sl_vlin.value = 0.0
    sl_vang.value = 0.0
btn_stop.on_click(on_stop)
widgets.VBox([sl_vlin, sl_vang, btn_stop])
```

Resultado: interface gráfica diretamente na célula do Jupyter!

📓 EXECUTE NO: NB-03, SEÇÃO 3.1

# ObstacleNet — IA para Robótica

**Rede neural que detecta obstáculos (NB-03, Seção 3.3)**

Arquitetura: Entrada (4 sensores) → Linear(32)+ReLU+Dropout → Linear(16)+ReLU → Linear(1)+Sigmoid

Tabela: Feature | Descrição

| Feature | Descrição |
|---|---|
| dist_frente | Distância frontal (m) |
| dist_esq | Distância esquerda (m) |
| dist_dir | Distância direita (m) |
| velocidade | Velocidade atual (m/s) |



Saída:

- Maior que 0.5 → Obstáculo detectado
- Menor ou igual a 0.5 → Caminho livre

Treinamento: 60 épocas, Adam lr=1e-3, BCELoss, acurácia alvo >= 80%

📓 Execute no: NB-03, Seção 3.3

# Módulo 4: Atividade Avaliativa

**PatrolNode — Robô autônomo em patrulha quadrada**

O robô deve:

1. Andar em linha reta por 2 metros
2. Girar 90 graus (aprox. 2 segundos a 0.785 rad/s)
3. Repetir 4 vezes para completar um quadrado
4. Interromper imediatamente se detectar obstáculo

Máquina de estados: MOVING_FORWARD → (dist OK) → TURNING → (90° completo) → MOVING_FORWARD; AVOIDING ← (obst!); (livre) ← AVOIDING; DONE quando N=4 lados

📓 NB-04 — Atividade Avaliativa

🔗 github.com/esp004-ppgesp/notebooks/blob/main/nb04_atividade_avaliativa.ipynb

# Rubrica de Avaliação

**Como sua nota é calculada (100 pontos)**

Tabela: Critério | Pontos

| Critério | Pontos |
|---|---|
| Ambiente configurado e NB-01 sem erros | 10 |
| NB-02: HelloNode + OuvidorNode funcionando | 15 |
| NB-02: VelocidadeNode com parâmetros | 10 |
| NB-02: Exercício QuadradoNode implementado | 15 |
| NB-03: Painel ipywidgets controlando nó | 15 |
| NB-03: ObstacleNet treinada (acurácia >= 80%) | 15 |
| NB-04: PatrolNode com máquina de estados | 15 |
| NB-04: Integração com ObstacleNet | 5 |
| **Total** | **100** |

# Google Colab: Alternativa em Nuvem

**Para quem ainda não tem Ubuntu instalado**

Como usar rosbags no Colab (análise de dados):

```
!pip install rosbags
from rosbags.rosbag2 import Reader
with Reader('/content/my_bag') as reader:
    for connection, timestamp, rawdata in reader.messages():
        print(connection.topic, timestamp)
```

Limitação: só leitura de bags offline — não controla robô em tempo real

Quando usar: análise de dados gravados, experimentos sem hardware

📓 Referência: NB-04, Seção 4.4

# Git e GitHub para a Disciplina

**Organize e versione seu trabalho**

Configuração inicial (uma vez só):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

Chave SSH para autenticação segura:

```
ssh-keygen -t ed25519 -C "seu@email.com"
```

Clonar o repositório:

```
git clone
git@github.com:esp004-ppgesp/notebooks.git
```

Fluxo de trabalho: editar arquivo → git add → git commit -m "feat: adiciona HelloNode" → git push

📓 INSTRUÇÕES COMPLETAS: NB-04, SEÇÃO 4.3

# Roteiro da Aula de Hoje

**O que faremos agora (passo a passo)**

| | |
|---|---|
| 01 | Clone o repositório da disciplina (5 min) |
| 02 | Abra o NB-01 e execute todas as células — verifique se tudo está OK (15 min) |
| 03 | Abra o NB-02 e execute os Exemplos 1 a 3 (20 min) |
| 04 | Discussão: dúvidas sobre nós, tópicos e mensagens (10 min) |
| 05 | Exemplo 4 e 5 do NB-02 — parâmetros e cinemática (15 min) |
| 06 | Para casa: exercício QuadradoNode (Exemplo Guiado do NB-02) |

🔗 TODOS OS LINKS: GITHUB.COM/ESP004-PPGESP/NOTEBOOKS

# Dúvidas Frequentes

## "O ROS 2 só funciona no Linux?"

→ Sim, usamos Ubuntu 24.04. Windows é suportado experimentalmente, mas não recomendado.

## "Posso usar WSL2 no Windows?"

→ Funciona parcialmente, mas comunicação de rede com robôs reais tem limitações.

## "O Jupyter não encontra rclpy. O que fazer?"

→ Verifique se está usando o kernel Python 3 (ROS2 Jazzy), não o kernel padrão.

## "Posso usar ROS 2 no Google Colab?"

→ Para rosbags e análise de dados: sim. Para controle em tempo real: não.

# Próximos Passos

**Continue aprendendo**

Recursos oficiais:

- Documentação ROS 2 Jazzy: docs.ros.org/en/jazzy
- Tutorials oficiais: docs.ros.org/en/jazzy/Tutorials.html
- rclpy API: docs.ros2.org/jazzy/api/rclpy

Comunidade:

- ROS Discourse: discourse.ros.org
- ROS Answers: answers.ros.org

Repositório da disciplina:

- github.com/esp004-ppgesp/notebooks — todos os notebooks, apostila e dados



Próxima aula: NB-03 — Controle interativo e treinamento da ObstacleNet